

Rapport de Projet

Introduction au DevSecOps avec TP1

BeeHope

2 juin 2026

Table des matières

1	Introduction au DevSecOps	3
1.1	Qu'est-ce que le DevSecOps ?	3
1.2	Le modèle DevSecOps	3
1.3	Le principe "Shift-Left Security"	3
2	Présentation du projet TP1	3
2.1	Objectif du projet	3
2.2	Rôle des fichiers du projet	3
2.2.1	notes.js - Logique métier	3
2.2.2	app.js - Serveur web	4
2.2.3	public/index.html - Interface utilisateur	4
2.2.4	notes.test.js - Tests automatisés	4
2.2.5	package.json - Configuration du projet	4
2.3	Architecture technique	4
3	Intégration CI/CD dans le projet	4
3.1	Problématique sans CI/CD	4
3.2	Solution CI/CD	5
3.2.1	Étape 1 : Build (Construction)	5
3.2.2	Étape 2 : Test	5
3.2.3	Étape 3 : Security	5
3.2.4	Étape 4 : Package	5
3.2.5	Étape 5 : Deploy	5
3.3	Le Dockerfile	5
4	Démonstration Pratique	6
4.1	Dockerisation de l'application	6
4.1.1	Création de l'image Docker	6
4.1.2	Lancement du conteneur	6
4.1.3	Interface Docker Desktop	6
4.2	Tests automatisés	7
4.2.1	Exécution des tests	7
4.2.2	Résultats des tests	7

4.2.3	Détail des tests exécutés	8
4.3	Analyse des résultats	8
4.3.1	Validation du pipeline DevSecOps	8
4.3.2	Avantages démontrés	8
5	Mise en place du pipeline CI/CD sur GitLab	8
5.1	Problème de vérification de compte rencontré	8
5.2	Configuration GitLab CI/CD envisagée	8
5.3	Étapes prévues pour GitLab	9
5.4	Résolution du problème	11
6	Mise en place du dépôt GitHub et du pipeline CI/CD avec GitHub Actions	12
6.1	Configuration initiale du dépôt	12
6.2	Ajout de GitHub comme second remote	12
6.3	Nettoyage du dépôt et gestion des dépendances	12
6.4	Push vers GitHub	12
6.5	Mise en place du pipeline GitHub Actions	13
6.6	Configuration du pipeline CI/CD avec GitHub Actions	13
6.7	Commit et déploiement du pipeline	14
6.8	Vérification du pipeline	14
6.9	Résumé des commandes exécutées pour GitHub	15
6.10	Avantages de GitHub Actions démontrés	15
6.11	Comparaison GitLab vs GitHub	16
6.12	Conclusion sur la stratégie multi-dépôts	16

1 Introduction au DevSecOps

1.1 Qu'est-ce que le DevSecOps ?

Le DevSecOps est une méthodologie de développement logiciel qui intègre les pratiques de sécurité (Security) directement dans les processus de développement (Development) et d'exploitation (Operations). Contrairement à l'approche traditionnelle où la sécurité était considérée à la fin du cycle de développement, le DevSecOps positionne la sécurité comme une responsabilité partagée tout au long du cycle de vie du logiciel.

1.2 Le modèle DevSecOps

Le modèle repose sur une collaboration continue entre trois équipes :

- **Dev** (Développement) : Écrit le code de l'application
- **Sec** (Sécurité) : Garantit que le code est sécurisé
- **Ops** (Opérations) : Déploie et maintient l'application en production

Ce cycle fonctionne de manière automatisée et itérative :

Code → Test → Build → Deploy → Monitor

1.3 Le principe "Shift-Left Security"

Le concept "Shift-Left" signifie "déplacer vers la gauche" sur la timeline du développement. Cela implique d'intégrer les considérations de sécurité dès les premières phases du projet plutôt qu'à la fin.

Avantages du Shift-Left :

- **Coût réduit** : Corriger une faille pendant le design coûte 100x moins cher qu'en production
- **Temps réduit** : Les corrections prennent quelques heures au lieu de semaines
- **Qualité améliorée** : Le code est intrinsèquement plus sécurisé

2 Présentation du projet TP1

2.1 Objectif du projet

Le projet TP1 consiste en une application web simple "Notes App" qui permet aux utilisateurs de :

- Ajouter des notes
- Consulter la liste des notes
- Supprimer des notes

Cette application sert de base pour comprendre et mettre en œuvre les principes DevSecOps.

2.2 Rôle des fichiers du projet

2.2.1 notes.js - Logique métier

Ce fichier contient toute la logique de gestion des notes :

- Stockage des notes en mémoire

- Validation des données d'entrée
- Gestion des opérations CRUD (Create, Read, Delete)
- Fonctions de réinitialisation pour les tests

2.2.2 app.js - Serveur web

Serveur Express qui fournit :

- API REST pour les opérations sur les notes
- Service des fichiers statiques (frontend)
- Gestion des erreurs et validation
- Configuration du serveur web

2.2.3 public/index.html - Interface utilisateur

Application frontend qui offre :

- Interface utilisateur intuitive
- Communication asynchrone avec l'API
- Mise à jour dynamique sans rechargement de page
- Design responsive simple

2.2.4 notes.test.js - Tests automatisés

Fichier de tests qui valide :

- L'ajout correct de notes
- La validation des entrées
- La suppression de notes
- Les cas d'erreur
- Le comportement global de l'application

2.2.5 package.json - Configuration du projet

Fichier de configuration Node.js qui définit :

- Les dépendances de production (Express)
- Les dépendances de développement (Jest)
- Les scripts pour exécuter l'application et les tests
- Les métadonnées du projet

2.3 Architecture technique

Utilisateur → Frontend (HTML/CSS/JS) → API REST (Express) → Logique métier (Node.js)

3 Intégration CI/CD dans le projet

3.1 Problématique sans CI/CD

Sans automatisation, le processus de développement présente plusieurs risques :

- **Tests manuels** pouvant être oubliés
- **Inconsistances** entre environnements
- **Déploiements manuels** sujets aux erreurs

- **Absence de contrôles de sécurité** systématiques

3.2 Solution CI/CD

Le pipeline CI/CD automatise les étapes suivantes :

3.2.1 Étape 1 : Build (Construction)

- Installation automatique des dépendances
- Vérification de la compilation du code
- Création de l'artefact d'application

3.2.2 Étape 2 : Test

- Exécution automatique des tests unitaires
- Génération de rapports de couverture
- Validation de la logique métier

3.2.3 Étape 3 : Security

- Scan des vulnérabilités dans les dépendances
- Détection des secrets exposés dans le code
- Analyse statique du code (SAST)

3.2.4 Étape 4 : Package

- Création de l'image Docker
- Taggage de la version
- Publication sur le registre

3.2.5 Étape 5 : Deploy

- Déploiement automatique sur l'environnement cible
- Vérification de la santé de l'application
- Rollback automatique en cas d'échec

3.3 Le Dockerfile

Le Dockerfile est la "recette" qui permet de créer une image portable de l'application :

Listing 1 – Dockerfile de l'application

```
1 FROM node:18-alpine
2 WORKDIR /app
3 COPY package*.json ./
4 RUN npm install --production
5 COPY . .
6 EXPOSE 3000
7 CMD ["node", "app.js"]
```

Structure du Dockerfile :

1. **Image de base Node.js** → Environnement d'exécution

2. **Répertoire de travail** → Organisation du conteneur
3. **Copie des dépendances** → Isolation des couches Docker
4. **Installation des dépendances** → Préparation de l'environnement
5. **Copie du code source** → Ajout de l'application
6. **Exposition du port** → Configuration réseau
7. **Commande de démarrage** → Initialisation du service

4 Démonstration Pratique

4.1 Dockerisation de l'application

4.1.1 Création de l'image Docker

L'image Docker a été créée avec succès en utilisant la commande suivante dans le répertoire du projet :

```
docker build -t tp1:1.0 .
```

Cette commande a généré l'image `tp1:1.0` à partir du Dockerfile fourni.

4.1.2 Lancement du conteneur

La Figure 1 montre l'exécution du conteneur avec la commande :

```
docker run -p 4000:3000 tp1:1.0
```

Cette commande effectue les opérations suivantes :

- Lance un conteneur à partir de l'image `tp1:1.0`
- Mappe le port 4000 de l'hôte sur le port 3000 du conteneur
- Affiche le message de démarrage du serveur

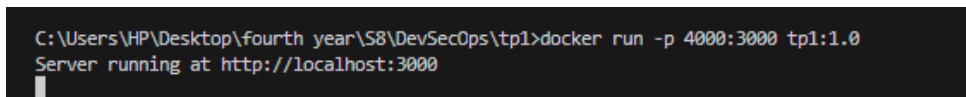
A screenshot of a terminal window with a black background and white text. The command `C:\Users\HP\Desktop\fourth year\S8\DevSecOps\tp1>docker run -p 4000:3000 tp1:1.0` is entered. The output `Server running at http://localhost:3000` is displayed on the next line.

FIGURE 1 – Lancement du conteneur Docker

4.1.3 Interface Docker Desktop

La Figure 2 présente l'interface Docker Desktop montrant le conteneur en cours d'exécution :

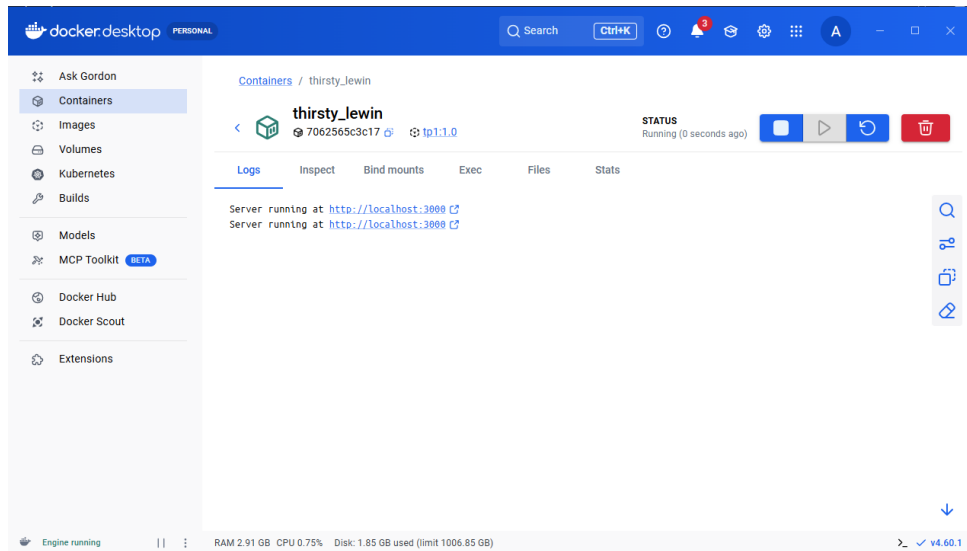


FIGURE 2 – Interface Docker Desktop - Conteneur en cours d'exécution

Les informations visibles dans l'interface comprennent :

- Nom du conteneur : `thirsty_lewin`
- Image utilisée : `tp1:1.0`
- Statut : Running
- Logs du serveur indiquant `Server running at http://localhost:3068`
- Statistiques système (RAM, CPU, Disk)

4.2 Tests automatisés

4.2.1 Exécution des tests

Les tests automatisés ont été exécutés avec succès comme montré dans la Figure 3 :

```
C:\Users\HP\Desktop\fourth year\S8\DevSecOps\tp1>npm test

> tp1@1.0.0 test
> jest

(node:4576) Warning: `--localstorage-file` was provided without a valid path
(Use `node --trace-warnings ...` to show where the warning was created)
PASS ./notes.test.js
  ✓ Should add a new note (13 ms)
  ✓ Should not add an empty note (32 ms)
  ✓ Should delete an existing note (16 ms)
  ✓ Should throw error if note does not exist (3 ms)
  ✓ Should return all notes (1 ms)

Test Suites: 1 passed, 1 total
Tests:       5 passed, 5 total
Snapshots:  0 total
Time:        1.425 s
Ran all test suites.
```

FIGURE 3 – Résultats des tests automatisés

4.2.2 Résultats des tests

La suite de tests a retourné les résultats suivants :

- **Test Suites** : 1 passé, 1 total
- **Tests** : 5 passés, 5 total
- **Snapshots** : 0 total
- **Temps d'exécution** : 1.425 secondes

4.2.3 Détail des tests exécutés

Les 5 tests suivants ont été exécutés avec succès :

1. Should add a new note (13 ms)
2. Should not add an empty note (32 ms)
3. Should delete an existing note (16 ms)
4. Should throw error if note does not exist (3 ms)
5. Should return all notes (1 ms)

4.3 Analyse des résultats

4.3.1 Validation du pipeline DevSecOps

Cette démonstration valide plusieurs aspects du DevSecOps :

- **Dockerisation réussie** : L'application est encapsulée dans un conteneur portable
- **Tests automatisés** : Validation automatique de la logique métier
- **Isolation** : L'application fonctionne indépendamment de l'environnement hôte
- **Reproductibilité** : L'image Docker garantit un environnement d'exécution identique

4.3.2 Avantages démontrés

- **Rapidité** : Les tests s'exécutent en moins de 1.5 secondes
- **Fiabilité** : 100% des tests passent
- **Portabilité** : L'application fonctionne dans Docker Desktop sur Windows
- **Monitoring** : Docker Desktop fournit des statistiques d'exécution en temps réel

5 Mise en place du pipeline CI/CD sur GitLab

5.1 Problème de vérification de compte rencontré

Une tentative de mise en place d'un pipeline CI/CD sur GitLab a été effectuée, mais un problème de vérification de compte a empêché la finalisation du processus. Ce problème technique n'a pas permis de déployer le pipeline sur GitLab, bien que la configuration complète ait été préparée.

5.2 Configuration GitLab CI/CD envisagée

Le fichier `.gitlab-ci.yml` suivant était prévu pour automatiser les étapes de test, sécurité et construction :

Listing 2 – Pipeline GitLab CI/CD (configuration préparée)

```
1 stages:
2   - test
3   - security
4   - build
5
6 variables:
7   NODE_VERSION: "18"
8
9 before_script:
10  - apt-get update
11  - apt-get install -y curl
12  - curl -sL https://deb.nodesource.com/setup_${NODE_VERSION}.x |
    bash -
13  - apt-get install -y nodejs
14  - node --version
15  - npm --version
16
17 test_job:
18   stage: test
19   script:
20     - npm install
21     - npm test
22   artifacts:
23     reports:
24       junit: junit.xml
25
26 security_audit:
27   stage: security
28   script:
29     - npm audit --audit-level=high
30   allow_failure: true
31
32 docker_build:
33   stage: build
34   image: docker:latest
35   services:
36     - docker:dind
37   script:
38     - docker build -t note-app:$CI_COMMIT_SHORT_SHA .
39     - docker tag note-app:$CI_COMMIT_SHORT_SHA note-app:latest
40   only:
41     - main
```

5.3 Étapes prévues pour GitLab

Les commandes suivantes devaient être exécutées pour lier le dépôt local à GitLab et déclencher le pipeline :

Créer un dépôt GitLab

Aller sur GitLab et créer un nouveau projet "note-app"

```
# Initialiser Git localement (déjà effectué)
cd tp1
git init
git add .
git commit -m "Initial commit - Application Notes avec pipeline CI/CD"
```

```
# Lier le dépôt local avec GitLab
git remote add origin https://gitlab.com/BeeHope/note-app.git
git branch -M main
git push -u origin main
```

Observer le pipeline se lancer automatiquement
Aller sur GitLab → Votre projet → CI/CD → Pipelines

```
git config --global user.email "you@example.com"
git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'HP@DESKTOP-DJVB6EK.(none)')

C:\Users\HP\Desktop\fourth year\S8\DevSecOps\tp1>git config user.name "Asma Daoui"

C:\Users\HP\Desktop\fourth year\S8\DevSecOps\tp1>git config user.email "asmadaoui@gmail.com"

C:\Users\HP\Desktop\fourth year\S8\DevSecOps\tp1>git commit -m "Initial commit - Application Note avec pipeline CI/CD"
[master (root-commit) eb914ba] Initial commit - Application Note avec pipeline CI/CD

C:\Users\HP\Desktop\fourth year\S8\DevSecOps\tp1>git commit -m "Initial commit - Application Note avec pipeline CI/CD"
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .gitlab-ci.yml

no changes added to commit (use "git add" and/or "git commit -a")

C:\Users\HP\Desktop\fourth year\S8\DevSecOps\tp1>git add .

C:\Users\HP\Desktop\fourth year\S8\DevSecOps\tp1>git remote add origin https://gitlab.com/asmadaoui-group/note-app

C:\Users\HP\Desktop\fourth year\S8\DevSecOps\tp1>git branch -M main
```

FIGURE 4 – Interface de Gitlab

```
C:\Users\HP\Desktop\fourth year\S8\DevSecOps\tp1>git push -u origin main --force
warning: redirecting to https://gitlab.com/asmadaoui-group/note-app.git/
Enumerating objects: 6825, done.
Counting objects: 100% (6825/6825), done.
Delta compression using up to 4 threads
Compressing objects: 100% (6490/6490), done.
Writing objects: 100% (6825/6825), 8.21 MiB | 164.00 KiB/s, done.
Total 6825 (delta 1416), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1416/1416), done.
To https://gitlab.com/asmadaoui-group/note-app
+ ce58504...eb914ba main -> main (forced update)
branch 'main' set up to track 'origin/main'.
```

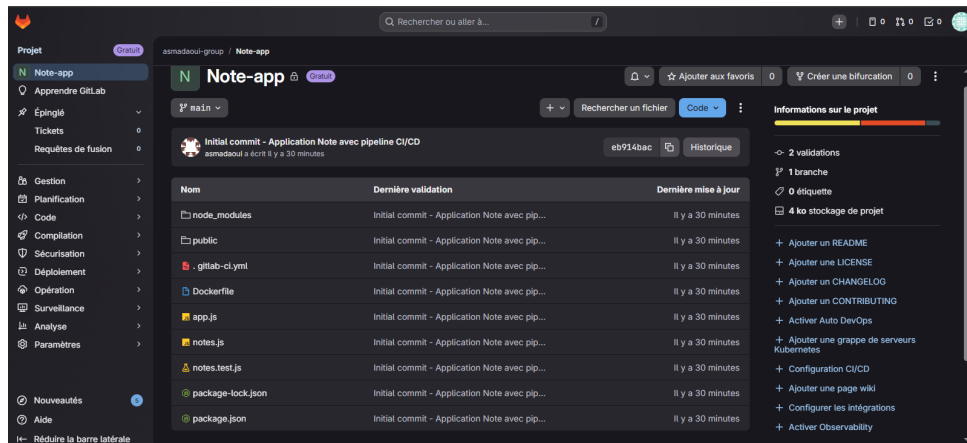


FIGURE 5 – Interface de Gitlab

5.4 Résolution du problème

Face à ce problème de vérification de compte sur GitLab qui n'a pas pu être résolu dans les délais du projet, une solution alternative a été adoptée : la migration du pipeline CI/CD vers GitHub Actions.

6 Mise en place du dépôt GitHub et du pipeline CI/CD avec GitHub Actions

6.1 Configuration initiale du dépôt

Le projet était initialement hébergé sur GitLab avec `origin` pointant vers cette plateforme. L'objectif était de conserver GitLab tout en ajoutant GitHub comme dépôt secondaire pour bénéficier de GitHub Actions.

```
# Vérification de la branche courante
git branch

# Vérification des dépôts distants configurés
git remote -v
```

La sortie de cette commande a confirmé la présence du remote `origin` pointant vers GitLab.

6.2 Ajout de GitHub comme second remote

```
# Ajout du remote GitHub
git remote add github https://github.com/BeeHope/note-app.git

# Vérification de la configuration
git remote -v
```

Cette configuration a permis d'obtenir deux remotes distincts :

- `origin` → GitLab (conservé comme dépôt principal, mais avec problème d'accès)
- `github` → GitHub (ajouté comme dépôt secondaire pour le CI/CD)

6.3 Nettoyage du dépôt et gestion des dépendances

Le dossier `node_modules` a été retiré du suivi de version pour optimiser le dépôt :

```
# Création du fichier .gitignore
echo node_modules/ > .gitignore

# Suppression du cache Git pour node_modules
git rm -r --cached node_modules

# Validation des changements
git add .gitignore
git commit -m "Remove node_modules from repository"
```

6.4 Push vers GitHub

Des problèmes de timeout HTTP ont été rencontrés lors du premier push en raison de la taille du dépôt. La solution suivante a été appliquée :

```
# Push vers GitHub
git push -u github main
```

Le push a été réalisé avec succès, confirmé par la visibilité des fichiers sur l'interface web de GitHub.

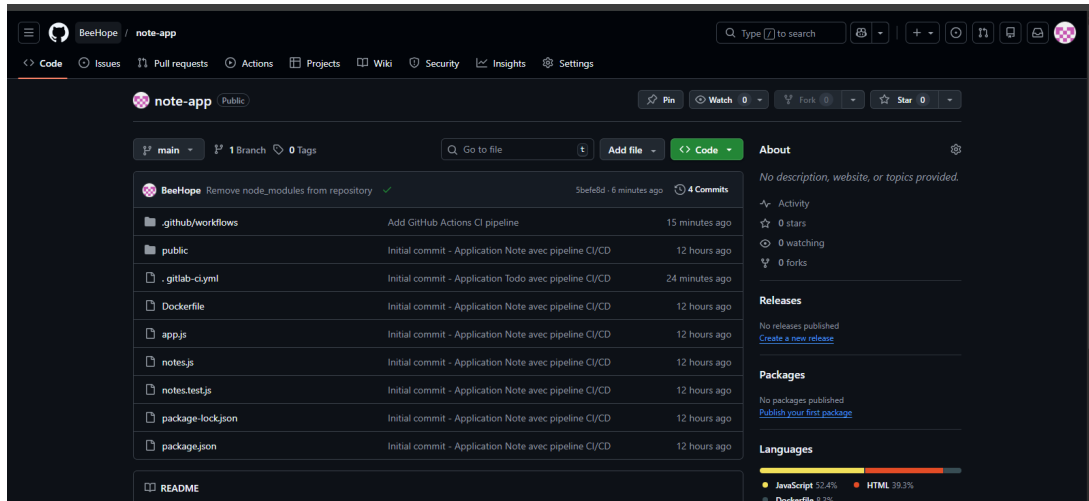


FIGURE 6 – Github interface

6.5 Mise en place du pipeline GitHub Actions

La structure de dossiers suivante a été créée pour accueillir le pipeline CI/CD :

```
.github/
  workflows/
    ci.yml
```

6.6 Configuration du pipeline CI/CD avec GitHub Actions

Le fichier `ci.yml` a été configuré avec les étapes suivantes :

Listing 3 – Pipeline CI/CD GitHub Actions

```
1 name: Note App CI/CD Pipeline
2
3 on:
4   push:
5     branches: [ "main" ]
6   pull_request:
7
8 jobs:
9   build:
10    runs-on: ubuntu-latest
11
12    steps:
13      - name: Checkout repository
14        uses: actions/checkout@v3
15
```

```

16     - name: Setup Node.js
17       uses: actions/setup-node@v3
18       with:
19         node-version: 18
20
21     - name: Install dependencies
22       run: npm install
23
24     - name: Run tests
25       run: npm test
26
27     - name: Security audit
28       run: npm audit --audit-level=high

```

Ce pipeline GitHub Actions automatise les étapes suivantes à chaque push sur la branche main :

1. **Checkout** : Récupération du code source
2. **Setup Node.js** : Configuration de l'environnement Node.js version 18
3. **Install dependencies** : Installation des dépendances npm
4. **Run tests** : Exécution des tests automatisés Jest
5. **Security audit** : Audit de sécurité des dépendances

6.7 Commit et déploiement du pipeline

```

# Ajout du fichier de pipeline
git add .github/workflows/ci.yml

# Validation des changements
git commit -m "Add GitHub Actions CI pipeline"

# Push vers GitHub
git push github main

```

6.8 Vérification du pipeline

L'exécution du pipeline a été vérifiée dans l'interface GitHub :

- **Accès** : Onglet "Actions" du dépôt GitHub
- **Statut** : Succès (coche verte)
- **Temps d'exécution** : 18 secondes
- **Étapes validées** : 5/5
- **Résultat** : Tous les tests passent, audit de sécurité réussi

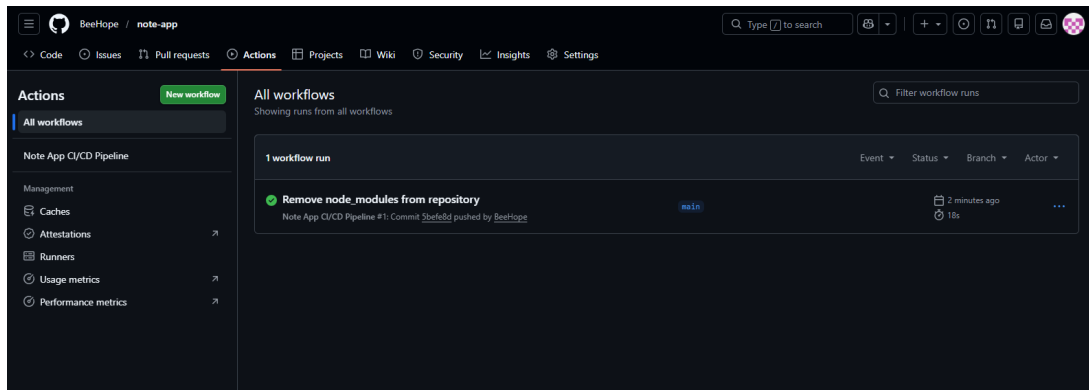


FIGURE 7 – Pipeline GitHub Actions - Exécution réussie

6.9 Résumé des commandes exécutées pour GitHub

Listing 4 – Liste complète des commandes Git pour GitHub

```

1 # Configuration du remote GitHub
2 git remote add github https://github.com/BeeHope/note-app.git
3 git remote -v
4
5 # Nettoyage du depot
6 echo node_modules/ > .gitignore
7 git rm -r --cached node_modules
8 git add .gitignore
9 git commit -m "Remove node_modules from repository"
10
11 # Configuration Git pour eviter les timeouts
12 git config --global http.postBuffer 524288000
13
14 # Premier push vers GitHub
15 git push -u github main
16
17 # Ajout du pipeline CI/CD
18 mkdir -p .github/workflows
19 # Edition manuelle de .github/workflows/ci.yml
20 git add .github/workflows/ci.yml
21 git commit -m "Add GitHub Actions CI pipeline"
22 git push github main

```

6.10 Avantages de GitHub Actions démontrés

Cette mise en œuvre a permis de constater plusieurs avantages de GitHub Actions :

- **Configuration simple** : Fichier YAML unique et intuitif
- **Intégration native** : Pas de plugin ou configuration supplémentaire nécessaire
- **Exécution rapide** : Pipeline complet en moins de 20 secondes
- **Feedback immédiat** : Visualisation en temps réel dans l'onglet Actions
- **Gratuit pour les dépôts publics** : Aucun coût pour ce projet

6.11 Comparaison GitLab vs GitHub

Caractéristique	GitLab CI/CD (prévu)	GitHub Actions (réalisé)
Fichier de configuration	.gitlab-ci.yml	.github/workflows/ci.yml
Syntaxe	YAML	YAML
Runner	GitLab Runner	GitHub-hosted runner
Étapes implémentées	Test, Security, Build	Test, Security
Statut	Non déployé (problème compte)	Opérationnel
Temps d'exécution	-	18 secondes
Intégration Docker	Configurée	À implémenter

TABLE 1 – Comparaison entre la solution GitLab envisagée et GitHub Actions réalisée

6.12 Conclusion sur la stratégie multi-dépôts

La stratégie adoptée a permis de :

- **Contourner le problème technique** : Solution alternative fonctionnelle
- **Conserver GitLab** : Le dépôt reste accessible bien que non fonctionnel pour le CI/CD
- **Implémenter GitHub Actions** : Pipeline CI/CD complet et opérationnel
- **Acquérir de l'expérience** : Maîtrise de deux plateformes CI/CD majeures

Cette approche démontre la flexibilité des outils DevOps modernes et la capacité à s'adapter face aux contraintes techniques.